Argentor

Secure Multi-Agent AI Framework in Rust

Technical Report

Version 0.1.0 — April 2026

3,400	~130K	15	14	40+
Tests Passing	Lines of Code	Crates	LLM Providers	Built-in Skills

0

Clippy Warnings

Table of Contents

1. Executive Summary
2. Architecture Overview
3. Crate Map
4. The Agentic Loop
5. Multi-Agent Orchestration
6. Security Architecture
7. WASM Plugin System
8. Protocol Support (MCP + A2A)
9. Built-in Skills (40+)
10. Gateway & REST API
11. Observability Stack
12. Compliance Modules
13. Data & Memory Layer
14. SDK & Language Bridges
15. Deployment
16. Project Metrics

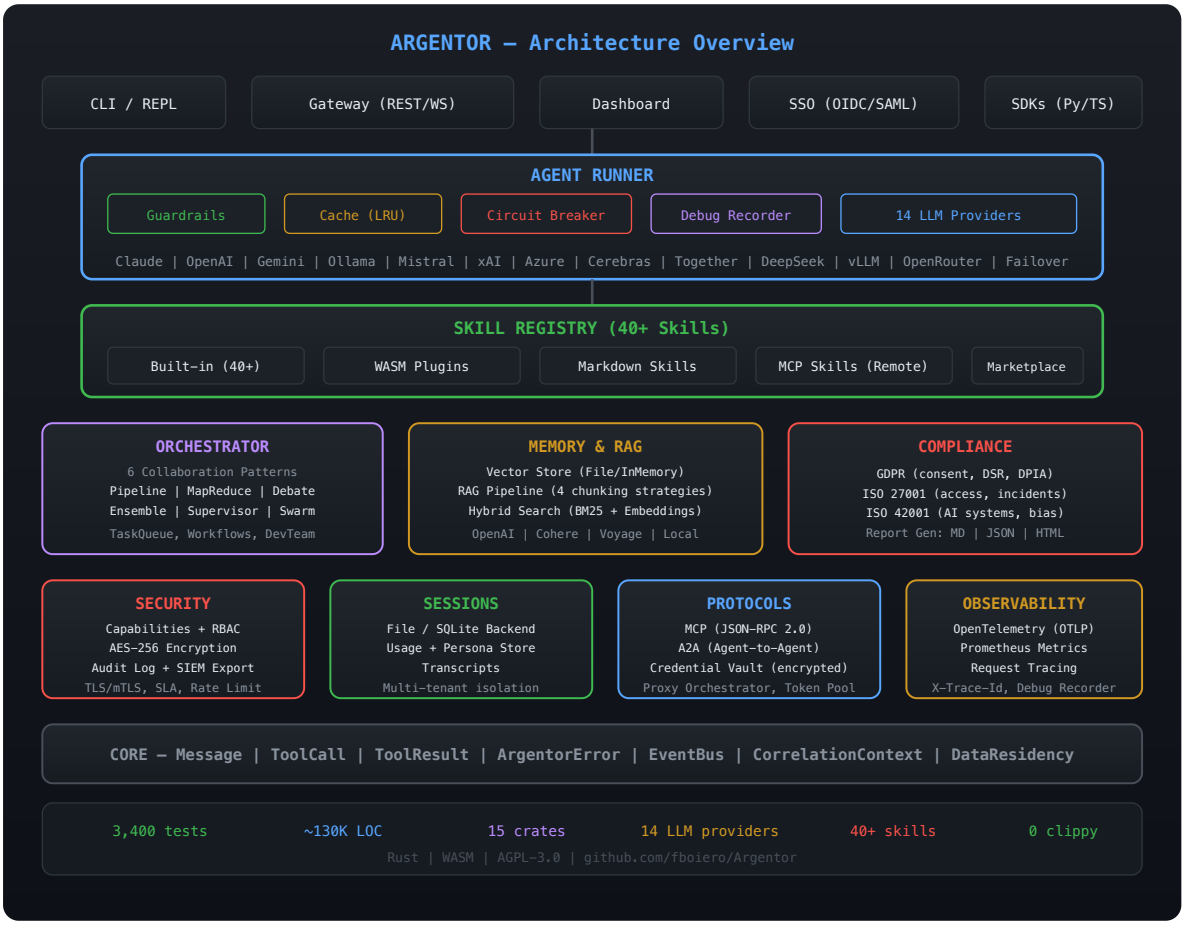
1. Executive Summary

Argentor is a production-grade multi-agent AI orchestration framework written in Rust. It provides:

- **15 crates** with clean dependency layering (zero circular deps)
- **14 LLM providers** (Claude, OpenAI, Gemini, Ollama, Mistral, xAI, Azure, Cerebras, Together, DeepSeek, vLLM, OpenRouter, Claude Code, Failover)
- **40+ built-in skills** covering data processing, web search, crypto, security scanning, file ops, code analysis
- **WASM-sandboxed plugins** with Ed25519 signing and capability-based permissions
- **MCP + A2A protocol** support for agent interoperability
- **Guardrails pipeline** integrated into the agentic loop (PII, injection, toxicity)
- **Enterprise compliance** modules (GDPR, ISO 27001, ISO 42001, DPGA)
- **3,400 tests**, zero failures, zero clippy warnings

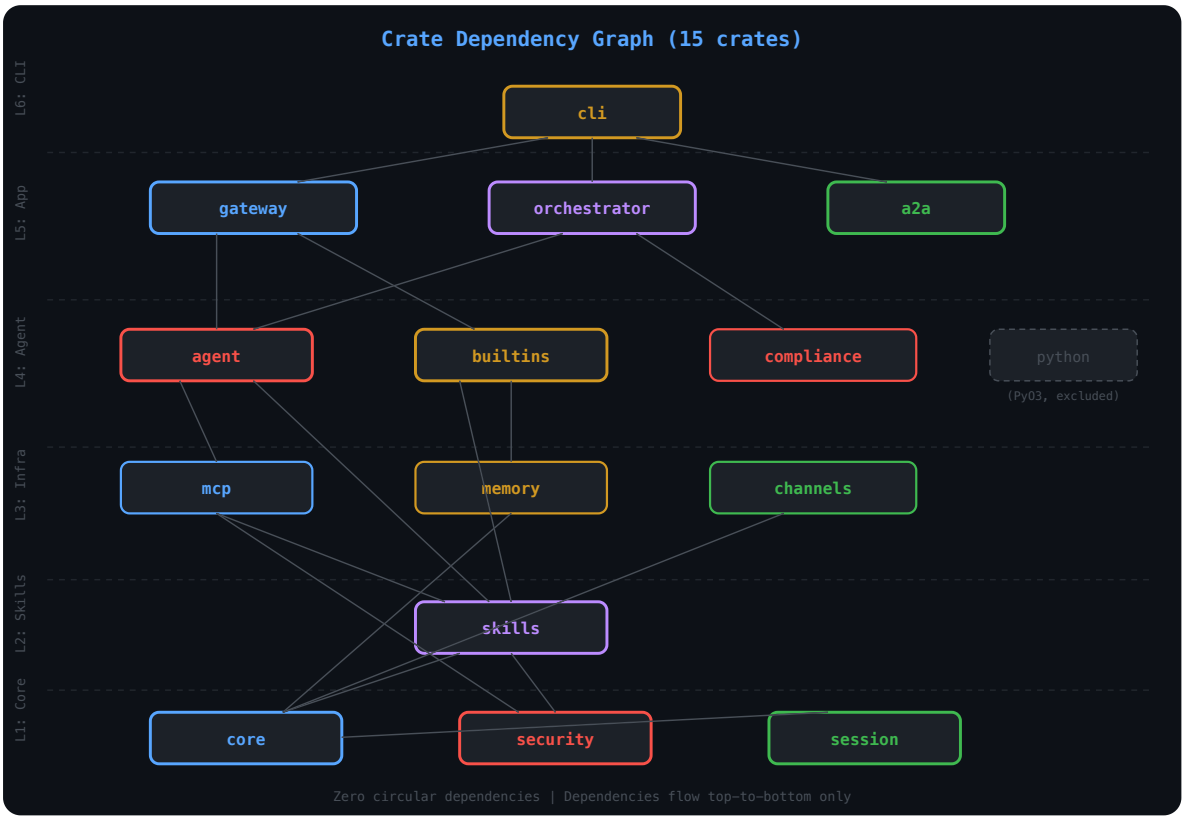
2. Architecture Overview

The framework is organized into six horizontal layers, from the entry points (CLI, Gateway, Dashboard, SSO) down through the Agent Runner, Skill Registry, Orchestrator, Memory and Compliance engines, to the foundational Core types.



3. Crate Map

Argentor is organized as a Cargo workspace of 15 crates with strict top-to-bottom dependency layering and zero circular dependencies.

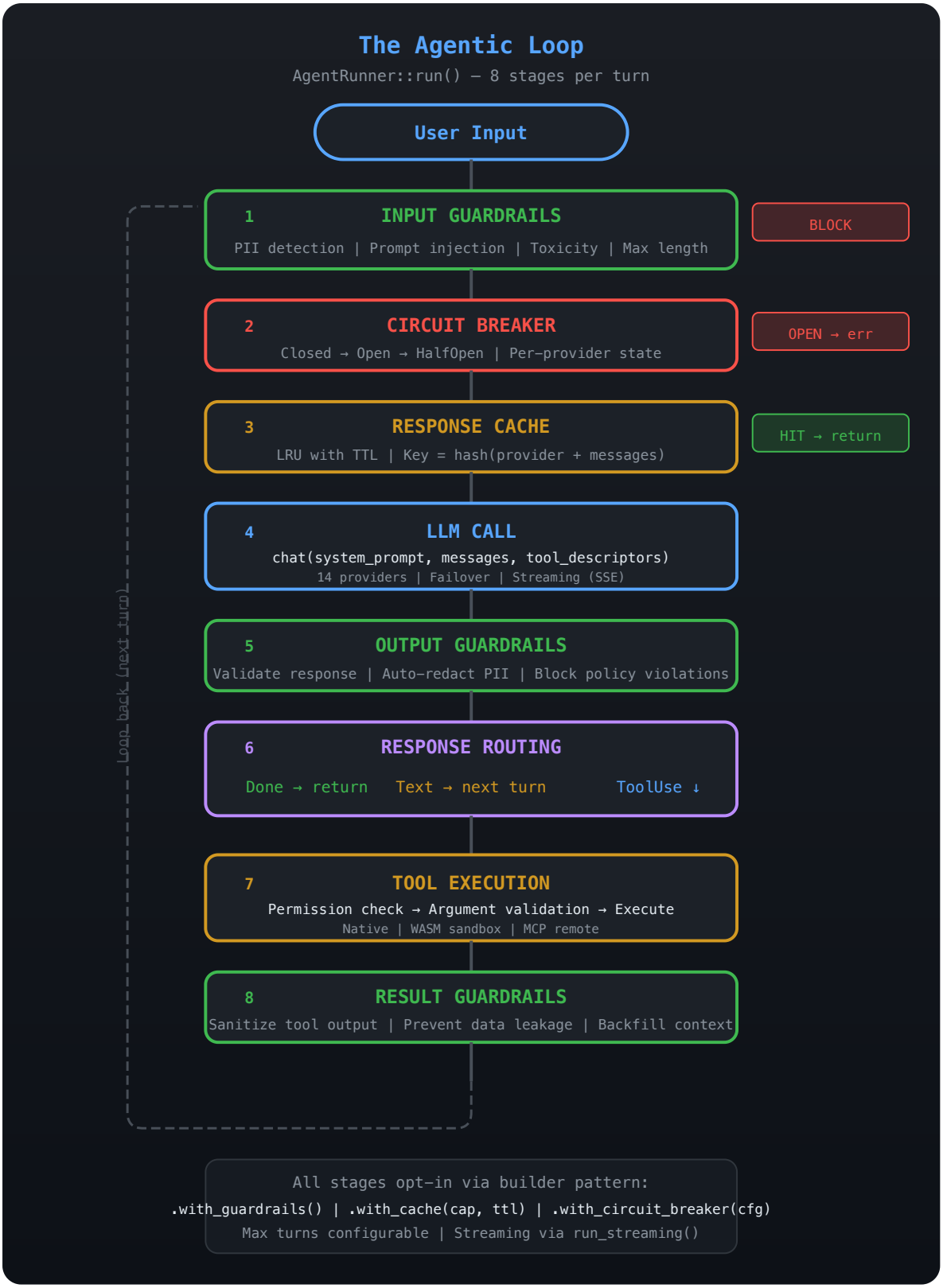


Crate Descriptions

#	Crate	LOC	Tests	Purpose
1	`argentor-core`	~8K	63	Types (Message, ToolCall, ToolResult), event bus, correlation, telemetry
2	`argentor-security`	~12K	526	Capabilities, RBAC, AES-256 encryption, audit log, SLA, rate limiting
3	`argentor-session`	~5K	84	Session lifecycle, SQLite backend, transcripts, usage/persona stores
4	`argentor-skills`	~8K	170	Skill trait, WASM runtime (wasmtime), registry, marketplace, vetting
5	`argentor-agent`	~18K	900	AgentRunner, 14 LLM backends, guardrails, ReAct, evaluator, cache
6	`argentor-channels`	~3K	18	Channel trait, Slack adapter, WebChat, manager
7	`argentor-gateway`	~20K	550	Axum HTTP/WS, REST API, SSO, observability, marketplace API
8	`argentor-builtins`	~25K	900	40+ built-in skills (all categories)
9	`argentor-memory`	~8K	140	Vector store, RAG pipeline, BM25, embeddings (3 providers)
10	`argentor-mcp`	~6K	126	MCP client/server, credential vault, token pool, proxy orchestrator
11	`argentor-orchestrator`	~15K	356	Multi-agent engine, 6 patterns, workflows, deployment, health
12	`argentor-compliance`	~7K	280	GDPR, ISO 27001, ISO 42001, DPGA, report generator
13	`argentor-a2a`	~4K	125	Google A2A protocol, agent cards, streaming SSE
14	`argentor-cli`	~5K	—	Binary, REPL, 8 demos, config
15	`argentor-python`	~1K	—	PyO3 bridge (9 classes, build with maturin)
	Total	**~130K**	**3,400**	

4. The Agentic Loop

The core execution engine (`AgentRunner::run()`) implements a multi-turn agentic loop with 8 stages:



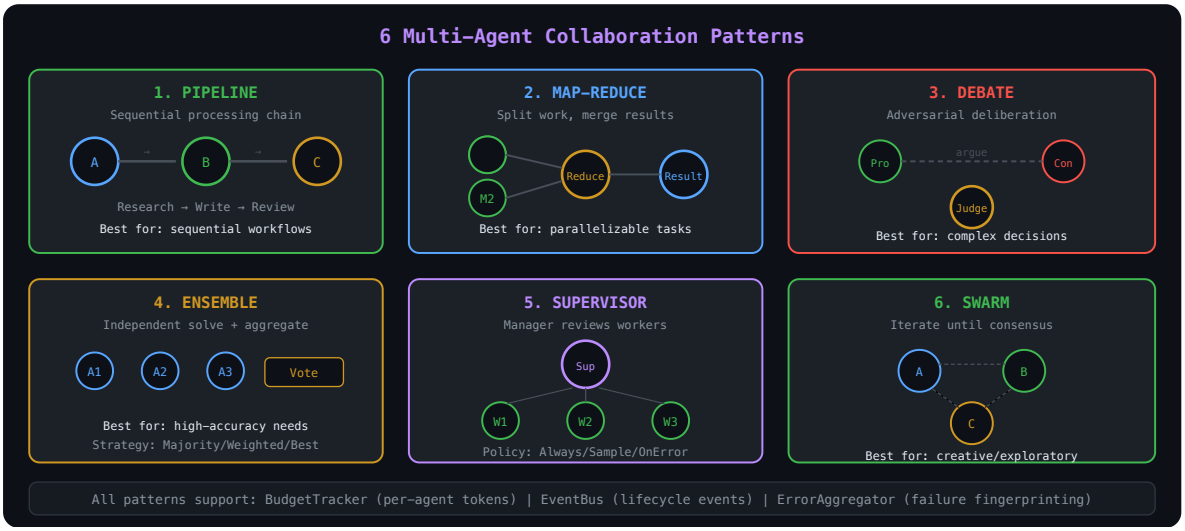
Key Design Decisions

- Guardrails run at 3 points: pre-LLM input, post-LLM output, post-tool result
- Cache operates BEFORE guardrails on output (cached responses already passed)
- Circuit breaker prevents wasting tokens on failing providers

- All stages are opt-in via builder pattern (`with_guardrails()`, `with_cache()`, etc.)

5. Multi-Agent Orchestration

The `Orchestrator` decomposes complex tasks into sub-tasks and distributes them across specialized worker agents. It supports 6 collaboration patterns, each optimized for different workload types.



6 Collaboration Patterns

Pattern	Best For	Example
Pipeline	Sequential workflows	Research → Write → Review
MapReduce	Parallelizable tasks	Analyze 100 files, merge results
Debate	Complex decisions	Security assessment with adversarial review
Ensemble	High-accuracy needs	Multiple agents vote on classification
Supervisor	Quality assurance	Manager reviews all worker outputs
Swarm	Creative/exploratory	Brainstorming with convergence

6. Security Architecture

Argentor implements defense in depth with 6 security layers, from authentication at the perimeter to encryption and audit at the data layer.

Security Layers Summary

- **Layer 1 — Authentication:** API Key, JWT/OAuth2, OIDC, SAML 2.0, Session Tokens
- **Layer 2 — Authorization:** RBAC with Admin, Developer, Analyst, Viewer roles; allow/deny per resource per action
- **Layer 3 — Capability-Based Permissions:** Skills declare capabilities (FileRead, FileWrite, ShellExec, NetworkAccess); PermissionSet grants least-privilege subset
- **Layer 4 — WASM Sandboxing:** Linear memory isolation, fuel limits, Ed25519 signing, SHA-256 checksums, static import analysis
- **Layer 5 — Guardrails Pipeline:** Pre-LLM input scan, post-LLM output validation, post-tool data leakage prevention

- **Layer 6 — Encryption & Audit:** AES-256-GCM vault, PBKDF2-HMAC-SHA256, TLS/mTLS, append-only audit log, SIEM export

7. WASM Plugin System

Third-party plugins are compiled to WebAssembly and executed in a sandboxed `wasmtime` runtime. Before loading, every plugin passes through the 5-check Skill Vetter.

Plugin Lifecycle

```
Developer                   Argentor Runtime
---------                   ----------------

skill.rs ----compile---> skill.wasm
                              |
                        SkillManifest
                          - name, version
                          - SHA-256 checksum
                          - Ed25519 signature
                          - capabilities[]
                          - tags, license
                              |
                        SKILL VETTER (5 checks)
                          1. SHA-256 checksum validation
                          2. Binary size limit (10MB max)
                          3. Ed25519 signature verification
                          4. Capability analysis (blocklist)
                          5. WASM static analysis
                             (magic number + import scanning)
                              | PASS
                        WASMTIME RUNTIME
                          - Linear memory isolation
                          - Fuel limits (CPU budget)
                          - WASI stdio only
                          - No filesystem/network access
                          - spawn_blocking() for async safety
```

Security Properties

- **Memory isolation:** Each WASM instance has its own linear memory; cannot access host memory
- **CPU budget:** Fuel limits prevent infinite loops and resource exhaustion
- **I/O restriction:** Only WASI stdio is available; no filesystem or network access
- **Code signing:** Ed25519 signatures verified before any execution
- **Static analysis:** Import scanning detects suspicious host function requests

8. Protocol Support (MCP + A2A)

Model Context Protocol (MCP)

Argentor acts as both MCP client and MCP server, enabling bidirectional interop with tools like Claude Desktop, Cursor, and other MCP-compatible agents.

```
Argentor MCP Client        JSON-RPC 2.0 (stdio)      External MCP Server
--------------------       --------------------       -------------------
methods:                                               Exposes:
  tools/list               <------------------->       - tools
  tools/call                                           - resources

External MCP Client        JSON-RPC 2.0 (HTTP)       Argentor MCP Server
--------------------       --------------------       -------------------
(Claude, Cursor, etc.)     <------------------->      Exposes all registered
                                                       skills as MCP tools
```

MCP Infrastructure:

- `McpProxy`: multiplexes calls to multiple MCP servers
- `CredentialVault`: AES-256 encrypted API key storage with rotation
- `TokenPool`: per-provider rate limiting with tier priority
- `ProxyOrchestrator`: routing (Fixed/RoundRobin/LeastLoaded/PatternBased), circuit breaker, failover

Agent-to-Agent Protocol (A2A)

```
Agent A (Argentor)         tasks/send                Agent B (Any A2A)
------------------         ----------                -----------------
Endpoints:                 ---------------->         /.well-known
  /a2a                     tasks/get                   /agent.json
  /a2a/stream              <----------------
                           SSE streaming
```

A2A Features:

- Agent discovery via `/.well-known/agent.json`
- Task lifecycle: submitted → working → completed/failed/canceled
- SSE streaming for real-time task updates
- Authentication: Bearer, API Key, OAuth2

9. Built-in Skills (40+)

Argentor ships with 40+ built-in skills organized into categories. Skills are registered in the `SkillRegistry` and exposed as tool descriptors to the LLM.

Category	Skills
Data & Text	`calculator`, `text_transform`, `json_query`, `regex`, `data_validator`, `datetime`
Web & Search	`web_search` (DDG/Tavily/Brave/SearXNG), `web_scraper`, `rss_reader`, `dns_lookup`
Code & Dev	`git`, `code_analysis`, `test_runner`, `sdk_generator`, `api_scaffold`, `iac_generator`
Memory	`memory_store`, `memory_search`
Crypto & Encoding	`hash` (SHA/HMAC), `encode_decode`, `uuid_generator`
File & System	`file_read`, `file_write`, `shell`, `http_fetch`
Security & AI	`prompt_guard`, `secret_scanner`, `diff`, `summarizer`
Orchestration	`agent_delegate`, `task_status`, `human_approval`, `artifact_store`
Browser & Docker	`browser_auto` (optional), `docker_sandbox` (optional)

Tool Groups (Progressive Disclosure)

Group	Skills	Use Case
`minimal`	echo, time, help, calculator, datetime	Safe for any context
`coding`	file_read/write, shell, memory, regex, json_query, diff	Development tasks
`web`	http_fetch, browser, web_search, web_scraper, rss_reader, dns_lookup	Web tasks
`data`	calculator, text_transform, json_query, regex, data_validator, encode_decode, hash, uuid, diff, summarizer	Data processing
`security`	prompt_guard, secret_scanner, hash, data_validator, encode_decode	Security scanning
`development`	git, code_analysis, test_runner, +coding tools	Full dev workflow
`orchestration`	agent_delegate, task_status, human_approval, artifact_store	Multi-agent
`full`	All registered skills	Unrestricted

10. Gateway & REST API

The gateway is built on **Axum** and provides a full REST API, WebSocket support, SSO authentication, and observability endpoints.

Middleware Stack

1. Request Tracing (X-Trace-Id, spans)
2. SSO Auth (Bearer token / cookie validation)
3. Per-Key Rate Limiting (sliding window, 429 + headers)
4. Auth (API key / JWT validation)

API Endpoints

Group	Method	Path	Description
Health	GET	`/health`	Basic health check
	GET	`/health/live`	Liveness probe
	GET	`/health/ready`	Readiness probe
Auth	GET	`/auth/login`	SSO redirect
	GET	`/auth/callback`	SSO callback
	GET	`/auth/logout`	Revoke session
	GET	`/auth/me`	Current user
	POST	`/auth/api-key`	Key to session
Agent API	POST	`/api/v1/agent/chat`	Chat completion
	POST	`/api/v1/agent/run-task`	Run agent task
	POST	`/api/v1/agent/run-task-stream`	SSE streaming
	POST	`/api/v1/agent/batch`	Batch processing
	POST	`/api/v1/agent/evaluate`	Evaluate agent
	GET	`/api/v1/agent/status`	Agent status
Sessions	GET	`/api/v1/sessions`	List sessions
	GET	`/api/v1/sessions/:id`	Get session
	DELETE	`/api/v1/sessions/:id`	Delete session
Skills	GET	`/api/v1/skills`	List skills
	GET	`/api/v1/skills/:name`	Get skill detail
Observability	GET	`/metrics`	Prometheus metrics
	GET	`/openapi.json`	OpenAPI 3.0 spec
	GET	`/dashboard`	HTML SPA dashboard
WebSocket	WS	`/ws`	Bidirectional messaging
A2A Protocol	GET	`/.well-known/agent.json`	Agent card
	POST	`/a2a`	JSON-RPC dispatch

	POST	`/a2a/stream`	SSE streaming

11. Observability Stack

Tracing (OpenTelemetry)

Instrumented with `#[tracing::instrument]` on all key methods:

- `AgentRunner::run()` — session_id, max_turns
- `AgentRunner::execute_tool()` — tool_name, id
- `AgentRunner::run_streaming()` — streaming=true
- LLM calls — provider, turn, session_id

Request middleware adds `X-Trace-Id` and `X-Span-Id` response headers. Export via OTLP gRPC to Jaeger, Tempo, or Datadog.

Metrics (Prometheus)

`GET /metrics` exposes:

Metric	Labels	Type
`http_requests_total`	method, path, status	Counter
`http_request_duration_seconds`	method, path	Histogram
`active_connections`	—	Gauge
`llm_calls_total`	provider	Counter
`tool_executions_total`	skill	Counter
`tokens_used_total`	provider, direction	Counter

Audit Log

Every action is recorded with: session_id, action, skill, outcome (Success/Denied/Error), and detailed JSON metadata. Export formats: Splunk, Elasticsearch, CEF, JSON-LD, Syslog.

Debug Recorder

Step-by-step trace of the agentic loop: Input → LlmCall → ToolCall → ToolResult → LlmResponse → Output, with timing, token counts, and metadata per step.

12. Compliance Modules

Four compliance frameworks are implemented as modules with automated report generation.

Framework	Components	Key Features
GDPR	ConsentStore, DSR Processing	Consent tracking, expiry, Data Subject Requests (Access, Erasure, Portability), 30-day SLA
ISO 27001	AccessControl, Incidents	Login events, denial rate tracking, incident severity/resolution lifecycle
ISO 42001	AI Inventory, Bias Checks, Transparency	Risk level assessment, model info, bias pass/warn/fail, decision log
DPGA	Digital Public Goods Alliance	9 indicators, open source verification, data privacy assessment

Report Generator

The compliance engine generates reports in three formats:

- **Markdown** — human-readable report
- **JSON** — API consumption
- **HTML** — dashboard rendering

Each report includes: overall compliance status (Compliant/Partial/NonCompliant), per-framework score (0–100%), critical findings, recommendations, and next review date.

13. Data & Memory Layer

RAG Pipeline

The retrieval-augmented generation pipeline follows the standard flow:

```
Ingest → Chunk → Embed → Store → Query → Format
```

Stage	Options
Chunking	FixedSize, Paragraph, Sentence, Semantic
Embedding	Local (TF-IDF), OpenAI, Cohere, Voyage
Search	Vector similarity + BM25 hybrid
Query Expansion	Rule-based synonym/related terms

Vector Stores

- `InMemoryVectorStore` — for development and testing
- `FileVectorStore` — persistent, trait-based, pluggable

Session Storage

- `FileSessionStore` — file-based session persistence
- `SQLite Backend` — WAL mode for concurrent access
- `JSON Index Store` — lightweight indexing

Conversation Memory

Cross-session per-customer context:

- `CustomerProfile` — topics, sentiment, interaction history
- `ConversationSummarizer` — token-budgeted summaries
- System prompt injection for seamless context continuity

14. SDK & Language Bridges

Python SDK (`pip install argentor-sdk`)

```
from argentor import ArgentorClient

client = ArgentorClient(base_url="http://localhost:8080", api_key="key")

# Run agent task
result = client.run_task(role="researcher", context="Analyze market trends")

# Stream responses
for event in client.run_task_stream(role="writer", context="Write a report"):
    print(event.text, end="")

# Execute skills directly
result = client.execute_skill("calculator", {"operation": "evaluate", "expressio

# Health check
health = client.health_ready()
```

TypeScript SDK (`npm install @argentor/sdk`)

```
import { ArgentorClient } from '@argentor/sdk';

const client = new ArgentorClient({
    baseUrl: 'http://localhost:8080',
    apiKey: 'key'
});

const result = await client.runTask({
    role: 'analyst',
    context: 'Summarize Q4 results'
});

for await (const event of client.runTaskStream({
    role: 'writer',
    context: 'Draft email'
})) {
    process.stdout.write(event.text);
}
```

PyO3 Bridge (`maturin develop`)

```python
import argentor

# Direct Rust execution from Python (no HTTP overhead)
registry = argentor.SkillRegistry()
result = registry.execute(
    "calculator",
    '{"operation": "evaluate", "expression": "sqrt(144)"}'
)
print(result.content)  # "12"

guard = argentor.GuardrailEngine()
check = guard.check_input("My SSN is 123-45-6789")
print(check.passed)          # False
print(check.sanitized_text)  # "My SSN is [SSN]"
```

15. Deployment

Docker Production

```yaml
# docker-compose.production.yml
services:
  argentor:
    build: .
    ports: ["8080:8080"]
    deploy:
      resources:
        limits: { memory: 512M, cpus: '2' }
    read_only: true
    cap_drop: [ALL]
    security_opt: [no-new-privileges:true]
    healthcheck:
      test: ["CMD", "wget", "--spider", "http://localhost:8080/health/live"]

  prometheus:   # optional (--profile monitoring)
    image: prom/prometheus:latest
    ports: ["9090:9090"]

  grafana:      # optional (--profile monitoring)
    image: grafana/grafana:latest
    ports: ["3000:3000"]
```

Helm Chart (Kubernetes)

```
deploy/helm/argentor/
  Chart.yaml
  values.yaml
  templates/
    deployment.yaml    (runAsNonRoot, readOnlyRootFilesystem, seccomp)
    service.yaml
    ingress.yaml
    hpa.yaml           (autoscaling 1 → 10 replicas)
    pvc.yaml
```

Security Hardening

- Read-only root filesystem
- All Linux capabilities dropped (`cap_drop: [ALL]`)
- No new privileges (`no-new-privileges:true`)
- Non-root user execution

- Seccomp profile enabled
- 512MB memory limit, 2 CPU cores
- HPA autoscaling from 1 to 10 replicas

16. Project Metrics

Metric	Value
Language	Rust (core) + WASM (plugins)
Crates	15 (14 workspace + 1 PyO3)
Lines of Code	~130,000
Tests	3,400 passing, 0 failures
Clippy	0 warnings with `-D warnings`
LLM Providers	14
Built-in Skills	40+
REST Endpoints	50+
Compliance Frameworks	4 (GDPR, ISO 27001, ISO 42001, DPGA)
Collaboration Patterns	6 (Pipeline, MapReduce, Debate, Ensemble, Supervisor, Swarm)
Feature Flags	7 (telemetry, sqlite, http-embeddings, registry, browser, docker, client)
License	AGPL-3.0-only
MSRV	1.80
SDKs	Python + TypeScript + PyO3

Release Channels

Channel	Package	Command
crates.io	13 crates	`cargo add argentor-core`
GitHub Releases	Binaries (Linux x86/arm, macOS x86/arm)	Download from releases
PyPI	`argentor-sdk`	`pip install argentor-sdk`
npm	`@argentor/sdk`	`npm install @argentor/sdk`
GHCR	`ghcr.io/fboiero/argentor`	`docker pull ghcr.io/fboiero/argentor`
GitHub Pages	Landing + demos	https://fboiero.github.io/Agentor

---

Generated for Argentor v0.1.0 — April 2026

Repository: https://github.com/fboiero/Argentor | License: AGPL-3.0-only